B7 Exploratory analysis done well

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B7 Exploratory analysis done well 3
B7-L1 Profile a dataset before you analyse it 3
B7-L2 Plot before you summarise 7
B7-L3 Write assumptions down, then turn them into checks 11
Track B best-practice checklist 16
Appendix: solutions and answer keys 17

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B7 Exploratory analysis done well

Roadmap: Prerequisite for all months. Profiling, visual checks and documented assumptions.

B7-L1 Profile a dataset before you analyse it

Concept

Before any chart or model, answer five questions about a new dataset. Each takes one line of pandas, and each catches a class of error that no later step will.

1. **Shape and grain.** How many rows, and what does one row mean? Check the claimed key is unique: `df.duplicated(subset=["respondent_id", "wave"]).sum()` should be 0.
2. **Dtypes.** `df.dtypes`. IDs read as floats, dates as text and numbers as `str` all show here (lesson B3-L1).
3. **Missingness.** `df.isna().mean()` gives the share missing per column. Look for columns that are nearly empty, and for shares that differ by wave or region.
4. **Cardinality.** `df.nunique()`. A `gender` column with 14 distinct values holds typos. An ID column with fewer distinct values than rows holds duplicates.
5. **Ranges.** `df.describe()`. Ages of 999, negative weights, and dates in 1900 are codes for "missing" or errors.

Put the per-column answers in one table:

```
def profile(df: pd.DataFrame) -> pd.DataFrame:
    return pd.DataFrame({
        "dtype": df.dtypes.astype(str),
        "missing_share": df.isna().mean(),
        "n_unique": df.nunique(),
    })
```

Two pandas defaults to know (pandas 3.0):

- `nunique()` **excludes** missing values. Pass `dropna=False` to count NaN as a value.
- `duplicated()` marks every repeat **except the first** (`keep="first"`). To count every row involved in a duplicate, use `keep=False`.

Save the profile with the analysis. When the next delivery arrives, compare the two profiles. A column whose missing share jumps from 2% to 40% tells you something changed at the source before it changes your results.

Libraries such as ydata-profiling build long HTML reports. They are useful for a first look. The five checks above, in code, are what you rerun on every delivery.

Why it matters

A satisfaction score was coded 1 to 10, with 99 for "don't know". Nobody profiled the file. The mean score for one region came out at 14.2, above the maximum possible value. The analyst assumed a weighting problem and spent a day on the weights. `describe()` would have shown a maximum of 99 in the first minute.

Worked example: weak code

```
df = pd.read_csv("wave12.csv")
df.head()
df.groupby("region")["score"].mean()
```

1. **Line 2.** Five rows show nothing about codes like 99, duplicates or missingness.
2. **Line 3.** Analysis before profiling: 99 codes are averaged in.

Worked example: strong code

```
import pandas as pd


def profile(df: pd.DataFrame) -> pd.DataFrame:
    """One row per column: dtype, missing share and distinct values."""
    return pd.DataFrame(
        {
            "dtype": df.dtypes.astype(str),
            "missing_share": df.isna().mean(),
            "n_unique": df.nunique(),
        }
    )


def duplicate_rows(df: pd.DataFrame, key: list[str]) -> int:
    """Rows whose key appears more than once, all copies counted."""
    return int(df.duplicated(subset=key, keep=False).sum())
```

1. **Line 9.** Share missing per column, comparable across deliveries.
2. **Line 17.** keep=False counts every copy, not only the repeats.

Shape and grain, dtypes, missingness, cardinality and ranges, in code, before any analysis.

Recap card

- **Five profiling questions** Shape and grain, dtypes, missingness, cardinality, ranges.
- **nunique() and missing values** Excluded by default. dropna=False counts NaN as a value.
- **Count every row in a duplicate group** df.duplicated(subset=key, keep=False).sum().
- **Why save the profile?** Compare it with the next delivery to spot source changes early.

Practice

Exercise 1. Multiple choice (level 1)

A satisfaction score should run from 1 to 10. Which call shows at once that some rows hold 99?

- a) `df.head()`
- b) `df["score"].describe()`
- c) `df.dtypes`
- d) `df.shape`

Why?

- r1) describe() reports min and max, so a max of 99 stands out.
- r2) describe() removes invalid values.

Exercise 2. Predict the output (level 2)

What does this print? (pandas 3.0)

```
import pandas as pd

ids = pd.Series(["a", "b", "b", None])
print(ids.nunique(), ids.duplicated().sum())
```

Exercise 3. Fill the gap (level 2)

Count every row that shares a (respondent_id, wave) key with another row, and the share of missing values per column.

```
dupes = df.duplicated(subset=["respondent_id", "wave"],
    keep=_____(1)).sum()
missing = df.isna()._____(2)()
```

Exercise 4. Write the function (level 3)

Write `profile(df: pd.DataFrame) -> pd.DataFrame`. Return one row per input column, indexed by column name, with columns:

- `dtype`: the dtype as a string.
- `missing_share`: share of missing values.
- `n_unique`: distinct non-missing values.
- `min` and `max`: for numeric columns, the minimum and maximum. For other columns, missing (`None` or `NaN`).

What you need to know.

Two pandas tools build this kind of summary:

- `pd.api.types.is_numeric_dtype(series)` is `True` for integer, float and boolean columns, and `False` for text.
- `pd.DataFrame.from_dict(rows, orient="index")` turns a dict of dicts into a frame. Each outer key becomes a row label, and each inner key becomes a column.

```
rows = {"a": {"n": 1, "kind": "x"}, "b": {"n": 2, "kind": "y"}}
pd.DataFrame.from_dict(rows, orient="index")
#    n kind
# a  1    x
# b  2    y
```

`series.isna().mean()` is the share of missing values, because `True` counts as 1.

Your code must pass these checks:

- Return one row per column, with columns dtype, missing_share, n_unique, min, max.
- missing_share is isna().mean(). n_unique excludes missing values.
- Numeric columns get min and max. Text columns get a missing value.
- dtype must be the column dtype as a string.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Profile each column: dtype, missing share, distinct values, numeric
    range.
def profile(df: pd.DataFrame) -> pd.DataFrame: ...
```

Starter code:

```
import pandas as pd


def profile(df):
```

```
    return df.describe()
```

Exercise 5. Spot the bug (level 4)

The QA report says 1 duplicated respondent row, but 2 rows in the file share the same ID, and both must be sent back to the supplier. Tap the line with the bug and fix it.

```
import pandas as pd


def rows_with_duplicate_ids(df: pd.DataFrame) -> pd.DataFrame:
    """Every row whose respondent_id appears more than once."""
    mask = df.duplicated(subset=["respondent_id"])
    return df[mask]
```

Write down the line number of the bug. Then write the fix.

Sources

- pandas nunique() excludes missing values by default and duplicated() defaults to keep="first". (verified: pandas API reference; run on pandas 3.0.6 and 3.0.2 in this repository.)

B7-L2 Plot before you summarise

Concept

Summary statistics hide shape. **Anscombe's quartet** (1973) is four small datasets with the same means, variances and correlation. Computed in this course's build for the first three:

Set	mean of y	variance of y	correlation with x
I	7.50	4.13	0.816
II	7.50	4.13	0.816
III	7.50	4.12	0.816

Plotted, set I is a noisy line, set II is a curve, and set III is a perfect line with one outlier. The numbers alone cannot tell them apart.

Four plots catch most problems:

- **Histogram** of each key numeric column. Shows skew, gaps, spikes at codes such as 99 or 0, and more than one peak.
- **Scatter plot** of each pair you will model or correlate. Shows curves, clusters and single points that drive a correlation.

- **Line over time** of counts and key rates by wave. Shows breaks in the series when a questionnaire or supplier changed.
- **Bar chart of missingness** by column and by subgroup.

Make each plot honest:

- Label both axes with units, and give every figure a title that states what it shows.
- Choose histogram bins on purpose. Too few hide a spike at 99. Try two or three bin counts.
- Use a log scale for heavily skewed values such as spend, and say so on the axis.
- Plot the raw data before you plot the smoothed or aggregated version.

Write plotting code as functions that return a `matplotlib.figure.Figure` (matplotlib 3.10). Build with `fig, ax = plt.subplots()`, not the global `plt.plot()` state. A function that returns a figure is testable: a test reads `ax.get_xlabel()` or counts the bars. The caller decides whether to show it or save it.

Why it matters

A team reported a correlation of 0.62 between ad recall and purchase intent across 40 markets, and the client planned a budget shift on it. A scatter plot showed 39 markets in a flat cloud and one market far in the top-right corner. Without that market the correlation was close to zero. The decision was about one market, not 40.

Worked example: weak code

```
❶ print(df["spend"].mean(), df["spend"].std())
❷ plt.hist(df["spend"])
❸ plt.show()
```

1. **Line 1.** Summaries first: a skewed spend column makes the mean misleading.
2. **Line 2.** Global pyplot state, default bins, no labels, no title.
3. **Line 3.** Nothing returned: nothing to test or save.

Worked example: strong code

```
1 import matplotlib.pyplot as plt
2 from matplotlib.figure import Figure
3
4
5 def plot_distribution(values: list[float], title: str, xlabel: str, bins:
    int = 30) -> Figure:
```

```
6     """Histogram of values with labelled axes. The caller shows or saves
      it."""
1     fig, ax = plt.subplots(figsize=(6, 4))
2     ax.hist(values, bins=bins)
9     ax.set_title(title)
10    ax.set_xlabel(xlabel)
11    ax.set_ylabel("Respondents")
3     return fig
```

1. **Line 7.** An explicit figure and axes, not the global pyplot state.
2. **Line 8.** Bins are a parameter, so you try several.
3. **Line 12.** Returned for the caller to show, save or test.

Plot distributions, pairs and time series before summarising, with functions that return labelled figures.

Recap card

- **Anscombe's quartet** Same mean, variance and correlation (0.816), different shapes. Plot before summarising.
- **Four first plots** Histograms, scatter plots of modelled pairs, lines over waves, missingness bars.
- **Testable plotting code** fig, ax = plt.subplots(), set labels on ax, return fig. No plt.show() inside.
- **Histogram bins** Choose on purpose and try several. Too few hide spikes at codes.

Practice

Exercise 1. Multiple choice (level 1)

Two variables have correlation 0.816 in four different datasets. What can you conclude about the relationship without plotting?

- a) It is linear in all four.
- b) It is the same shape in all four.
- c) Very little about its shape.
- d) An outlier is present in all four.

Why?

- r1) Anscombe's quartet has identical correlations with a line, a curve and an outlier-driven fit.
- r2) Correlation is always misleading.

Exercise 2. Predict the output (level 2)

What does this print? (NumPy 2.4)

What you need to know.

`np.corrcoef(x, y)` returns a 2 by 2 matrix of Pearson correlations: each variable with itself on the diagonal (always 1.0), and the correlation between `x` and `y` at `[0, 1]` and `[1, 0]`.

```
np.corrcoef([1, 2, 3], [2, 4, 7])[0, 1]   # 0.993...
```

```
import numpy as np

x = [10, 8, 13, 9, 11, 14, 6, 4, 12, 7, 5]
y1 = [8.04, 6.95, 7.58, 8.81, 8.33, 9.96, 7.24, 4.26, 10.84, 4.82, 5.68]
y3 = [7.46, 6.77, 12.74, 7.11, 7.81, 8.84, 6.08, 5.39, 8.15, 6.42, 5.73]
print(round(np.corrcoef(x, y1)[0, 1], 2) == round(np.corrcoef(x, y3)[0,
    1], 2))
```

Exercise 3. Fill the gap (level 2)

Create an explicit figure and axes, and label the x axis.

```
fig, ax = plt._____(1)()
ax.hist(values, bins=30)
ax._____(2)("Monthly spend (GBP)")
```

Exercise 4. Write the function (level 3)

Write `plot_distribution(values, title, xlabel, bins=30) -> Figure`. Draw a histogram with `bins` bins on one axes, set the title and x label from the arguments, set the y label to `"Respondents"`, and return the figure. Do not call `plt.show()`.

Your code must pass these checks:

- Return a matplotlib Figure.
- Set the title and x label from the arguments, and the y label to Respondents.
- Use the bins argument for the histogram.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Histogram of values with labelled axes.
def plot_distribution(
    values: list[float],
    title: str,
    xlabel: str,
    bins: int = 30,
) -> Figure: ...
```

Starter code:

```python
import matplotlib.pyplot as plt


def plot_distribution(values, title, xlabel, bins=30):
    plt.hist(values)
    plt.show()
```

Exercise 5. Architecture design (level 4)

Choose the first plot for each question.

Options: Histogram, Scatter plot, Line over waves, Bar chart of missing share.

1. Does the age column hold codes such as 999?
2. Is the recall and intent correlation driven by a few markets?
3. Did the supplier change in wave 9 shift the awareness rate?
4. Which questions did the new mobile layout leave unanswered?

Sources

- Anscombe's sets I to III share mean 7.50, variance about 4.12 to 4.13 and correlation 0.816. (verified: Anscombe (1973), The American Statistician 27(1); computed with NumPy 2.4.6 in this repository.)

B7-L3 Write assumptions down, then turn them into checks

Concept

Every analysis rests on assumptions: what a row means, how "don't know" is coded, which respondents are in scope, that weights sum to the sample size. Unwritten, they live in one analyst's head and break silently when the data changes.

Keep an **assumptions log** next to the analysis. One entry per assumption, with the same fields each time:

```yaml
- id: A3
  statement: "score uses 1 to 10; 98 = refused, 99 = don't know"
  evidence: "Questionnaire v4, Q12; profile of waves 1 to 11 shows only
   1-10, 98, 99"
  handling: "98 and 99 become missing before any statistic"
  check: "score in 1..10 or missing, after recoding"
  owner: "analysis team"
  last_verified: 2026-09-01
```

Then make the assumption **executable**. The `handling` becomes code, and the `check` becomes a QA result (lesson B5-L3) that runs on every delivery.

```
MISSING_CODES: dict[str, list[int]] = {"score": [98, 99]}
  # from config, per A3


def recode_missing(df: pd.DataFrame, codes: dict[str, list[int]]) ->
  pd.DataFrame:
    return df.assign(**{col: df[col].mask(df[col].isin(vals)) for col,
  vals in codes.items()})
```

Recode **per column**. `df.replace(99, np.nan)` on the whole frame also blanks a real age of 99. `Series.mask(cond)` sets values to missing where the condition holds and leaves the rest.

Three kinds of assumption to write down every time:

- **Structure.** Grain, keys, which columns are required.
- **Coding.** Missing codes, scales, units, category lists.
- **Scope.** Who is included: age limits, completes only, which markets and waves.

When a check fails, the log tells you which belief was wrong and who to ask. When a client asks "why is this respondent excluded?", the answer has an ID.

Why it matters

An analyst knew that 99 meant "don't know" on the satisfaction scale and removed those rows by hand in a notebook. The analyst left. The next wave went through the scripted pipeline, which did not know. The regional mean rose from 7.1 to 11.4. It was published, because 11.4 looked like a large improvement rather than an impossible value on a 1 to 10 scale.

A logged assumption with an executable check would have failed the run on the first 99.

Worked example: weak code

```
# 99 = don't know, drop them (ask Sam)
df = df[df["score"] != 99]
df = df.replace(98, np.nan)
```

1. **Line 1.** An assumption in a comment, tied to one person, with no check.
2. **Line 2.** Drops whole rows, so other answers from those respondents are lost too.
3. **Line 3.** Whole-frame replace: any column with a real 98 is blanked.

Worked example: strong code

```python
import pandas as pd


def recode_missing(df: pd.DataFrame, codes: dict[str, list[int]]) -> pd.DataFrame:
    """Set listed codes to missing, per column (assumption A3). Other columns untouched."""
    return df.assign(**{col: df[col].mask(df[col].isin(vals)) for col, vals in codes.items()})


def score_in_range(df: pd.DataFrame, low: int, high: int) -> bool:
    """Check for A3: every non-missing score lies in [low, high]."""
    s = df["score"].dropna()
    return bool(s.between(low, high).all())
```

1. **Line 6.** Per-column recoding from config. The respondent keeps their other answers.
2. **Line 12.** The assumption as a check, run on every delivery.

Log each assumption with evidence and owner, implement its handling per column, and check it on every run.

Recap card

- **Fields of an assumptions log entry** id, statement, evidence, handling, check, owner, last_verified.
- **Recode missing codes safely** Per column: df[col].mask(df[col].isin(codes)). Never a whole-frame replace.
- **Three kinds of assumption** Structure (grain, keys), coding (codes, scales, units), scope (who is included).
- **From assumption to code** Handling in config-driven code, and a QA check that runs on every delivery.

Practice

Exercise 1. Multiple choice (level 1)

Where should the assumption "98 and 99 mean refused and don't know on score" live?

- a) In a code comment where the rows are dropped.
- b) In the assumptions log, in config for the handling, and in a QA check.
- c) In the analyst's notes.
- d) Nowhere: everyone knows the convention.

Why?

- r1) Written, applied and checked in code, it survives staff changes and new deliveries.
- r2) Logs are required by law.

Exercise 2. Predict the output (level 2)

What does this print? (pandas 3.0)

```
import numpy as np
import pandas as pd

df = pd.DataFrame({"score": [3, 99, 7], "age": [99, 40, 30]})
print(df.replace(99, np.nan)["age"].tolist())
```

Exercise 3. Put in order (level 2)

Order the life of one assumption.

- Observe the coding in the questionnaire and the profile
- Implement the handling in code, driven by config
- Write the assumption, evidence and owner in the log
- Update last_verified when the check or the source is reviewed
- Add a QA check that runs on every delivery

Exercise 4. Write the function (level 3)

Write `recode_missing(df, codes) -> pd.DataFrame`. `codes` maps a column name to a list of values that mean "missing" in that column. Return a new frame where those values are missing in those columns only. Leave other columns and the input frame unchanged.

Your code must pass these checks:

- Listed codes in the listed column must become missing.
- Other columns must keep their values, including a real 99 or 98.
- The input frame changed. Return a new one.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Set listed codes to missing, per column.
def recode_missing(
    df: pd.DataFrame,
    codes: dict[str, list[int]],
) -> pd.DataFrame: ...
```

Starter code:

```
import numpy as np
```

```
import pandas as pd


def recode_missing(df, codes):
    for vals in codes.values():
        df = df.replace(vals, np.nan)
    return df
```

Exercise 5. Architecture design (level 4)

Classify each assumption and pick its check.

Options: Structure: check key uniqueness, Coding: check values against the codebook, Scope: check inclusion rules.

1. One row per respondent per wave
2. region uses codes 1 to 12 from the lookup table
3. Only respondents aged 18 or over are reported
4. Only completed interviews are in the file

Sources

- DataFrame.replace(99, np.nan) changes every column holding 99; Series.mask(cond) sets values where cond is True to missing. (verified: pandas API reference; run on pandas 3.0.6 in this repository.)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B7-L1 Profile a dataset before you analyse it

B7-L1 exercise 1

Hints. (1) Which call reports the maximum? (2) What would the maximum be? (3) 99.

Answer. b) `df["score"].describe()` Because: r1) describe() reports min and max, so a max of 99 stands out.

`describe()` gives count, mean, min, quartiles and max. A max of 99 on a 1 to 10 scale is a code, not a score.

Common wrong answers.

- `a`: head() shows five rows. The 99s are likely elsewhere.
- `c`: 99 is a valid integer. The dtype looks fine.
- `d`: The shape counts rows and columns, not values.
- `b|r2`: Right answer, wrong reason. describe() reports values. It does not clean them.

B7-L1 exercise 2

Hints. (1) Does nunique count None? (2) Which b does duplicated() mark? (3) keep="first" is the default.

Answer.

```
1 2 1
```

Two distinct non-missing values. One repeat, because the first `b` is not marked.

Common wrong answers.

- `3 1`: nunique() excludes missing values by default. Only a and b count.
- `2 2`: duplicated() keeps the first b unmarked. Only the second b is True.

B7-L1 exercise 3

Hints. (1) Which keep value marks every copy? (2) The mean of True/False is the share of True. (3) `False` and `mean`.

Answer.

```python
dupes = df.duplicated(subset=["respondent_id", "wave"], keep=False).sum()
missing = df.isna().mean()
```

`keep=False` flags all copies. `isna().mean()` gives the missing share per column.

Common wrong answers.

- g1 = `"first"`: keep="first" leaves one copy unmarked. keep=False marks all of them.
- g2 = `sum`: sum() gives counts. The mean of a boolean column is the share.

B7-L1 exercise 4

Hints. (1) Loop over columns and build a dict per column. (2) `pd.api.types.is_numeric_dtype(s)` tells you whether min and max apply. (3) `pd.DataFrame.from_dict(rows, orient="index")` builds the table.

Answer.

```python
import pandas as pd


def profile(df: pd.DataFrame) -> pd.DataFrame:
    """Profile each column: dtype, missing share, distinct values, numeric range.

    Args:
        df: Any DataFrame.

    Returns:
        One row per input column.
    """
    rows = {}
    for col in df.columns:
        s = df[col]
        numeric = pd.api.types.is_numeric_dtype(s)
        rows[col] = {
            "dtype": str(s.dtype),
            "missing_share": float(s.isna().mean()),
            "n_unique": int(s.nunique()),
            "min": float(s.min()) if numeric else None,
            "max": float(s.max()) if numeric else None,
        }
    return pd.DataFrame.from_dict(rows, orient="index")
```

A per-column table is easy to save and to compare with the next delivery's profile.

B7-L1 exercise 5

Hints. (1) Which rows does duplicated() mark by default? (2) Is the first b marked? (3) keep=False.

Answer. The bug is on line 6.

Fixed version:

```
1 import pandas as pd
2
3
4 def rows_with_duplicate_ids(df: pd.DataFrame) -> pd.DataFrame:
5     """Every row whose respondent_id appears more than once."""
6     mask = df.duplicated(subset=["respondent_id"], keep=False)
7     return df[mask]
```

The default `keep="first"` leaves the first copy unmarked. `keep=False` marks both.

Common wrong answers.

- Line 7: Filtering with the mask is right. The mask misses the first copy.

B7-L2 Plot before you summarise

B7-L2 exercise 1

Hints. (1) What does one number summarise? (2) Remember Anscombe. (3) Shape needs a plot.

Answer. c) Very little about its shape. Because: r1) Anscombe's quartet has identical correlations with a line, a curve and an outlier-driven fit.

Correlation measures linear association. Curves, clusters and single points give the same value. Plot first.

Common wrong answers.

- `a`: Anscombe's set II is a curve with the same correlation.
- `b`: Same correlation, different shapes: that is the point of the quartet.
- `d`: Only some of the sets have an outlier. The number cannot tell you which.
- `c|r2`: Right answer, wrong reason. Correlation is useful. It does not describe shape.

B7-L2 exercise 2

Hints. (1) These are Anscombe's sets I and III. (2) What correlation does the quartet share? (3) 0.816.

Answer.

```
True
```

Both correlations round to 0.82. Only a plot shows that set III is a perfect line bent by one point.

Common wrong answers.

- `False`: Both are 0.82 to two places. Set III is a straight line plus one outlier, and still matches set I.

B7-L2 exercise 3

Hints. (1) The pyplot function that returns (fig, ax). (2) Axes methods that set things start with set_. (3) `subplots` and `set_xlabel`.

Answer.

```
fig, ax = plt.subplots()
ax.hist(values, bins=30)
ax.set_xlabel("Monthly spend (GBP)")
```

`plt.subplots()` returns a figure and axes. Axes setters are `set_title`, `set_xlabel` and `set_ylabel`.

Common wrong answers.

- g2 = `xlabel`: plt.xlabel is the global pyplot function. On an Axes object the method is set_xlabel.

B7-L2 exercise 4

Hints. (1) Start with `fig, ax = plt.subplots()`. (2) `ax.hist(values, bins=bins)`, then the three set_ calls. (3) Return fig.

Answer.

```
import matplotlib.pyplot as plt
from matplotlib.figure import Figure


def plot_distribution(values: list[float], title: str, xlabel: str, bins:
    int = 30) -> Figure:
    """Histogram of values with labelled axes.

    Args:
        values: Numbers to plot.
        title: Figure title.
        xlabel: X axis label, with units.
        bins: Number of histogram bins.

    Returns:
        The figure. The caller shows or saves it.
```

```
    """
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.hist(values, bins=bins)
    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.set_ylabel("Respondents")
    return fig
```

Returning a labelled figure makes the plot testable and lets the caller choose to show or save it.

B7-L2 exercise 5

Hints. (1) How many variables does the question involve? (2) Is time part of the question? (3) Missingness is its own variable.

Answer.

- Codes and spikes: a histogram.
- A relationship and its outliers: a scatter plot.
- A break in a series: a line over waves.
- Where answers are missing: a bar chart of missing share by column and device.

Match the plot to the question: one variable, a pair, a series or missingness.

Common wrong answers.

- codes = `scatter`: A spike at 999 in one variable shows best as a tall, isolated bar in a histogram.
- driver = `hist`: A histogram shows one variable. Influential points need both variables: a scatter plot.
- break = `hist`: A histogram pools the waves. A line over waves shows the step at wave 9.

B7-L3 Write assumptions down, then turn them into checks

B7-L3 exercise 1

Hints. (1) What happens when the analyst leaves? (2) What happens when the supplier changes the code? (3) Written, applied, checked.

Answer. b) In the assumptions log, in config for the handling, and in a QA check. Because: r1) Written, applied and checked in code, it survives staff changes and new deliveries.

The log states it, config applies it, and a check detects when it stops being true.

Common wrong answers.

- `a`: A comment explains one line. It does not run, and nothing fails when the coding changes.
- `c`: Personal notes leave with the person.
- `d`: New analysts and new pipelines do not know. That is how 99s reach a mean.
- `b|r2`: Right answer, wrong reason. The point is surviving change, not compliance.

B7-L3 exercise 2

Hints. (1) Which columns does df.replace touch? (2) Is 99 a valid age? (3) It is blanked anyway, and the column becomes float.

Answer.

```
1 [nan, 40.0, 30.0]
```

Whole-frame `replace` blanks the 99-year-old too. Recode per column with a mapping of column to codes.

Common wrong answers.

- `[99, 40, 30]`: replace on the whole frame changes every column, including a real age of 99.

B7-L3 exercise 3

Hints. (1) Evidence before belief. (2) Belief before code. (3) Handling before the check that tests its result.

Answer.

1. Observe the coding in the questionnaire and the profile
2. Write the assumption, evidence and owner in the log
3. Implement the handling in code, driven by config
4. Add a QA check that runs on every delivery
5. Update last_verified when the check or the source is reviewed

Observe, write, implement, check, review. Each step leaves a record the next analyst can follow.

Common wrong answers.

- Code written before the evidence encodes a guess.
- The check runs on recoded data, so the handling comes first.

B7-L3 exercise 4

Hints. (1) Work column by column: `df[col].isin(vals)` marks the codes. (2) `Series.mask(cond)` sets values to missing where cond is True. (3) Wrap it in `df.assign(**{col: ... for col, vals in codes.items()})`.

Answer.

```python
import pandas as pd


def recode_missing(df: pd.DataFrame, codes: dict[str, list[int]]) -> pd.DataFrame:
    """Set listed codes to missing, per column.

    Args:
        df: Input frame.
        codes: Column name to the values that mean missing in that column.

    Returns:
        A new frame. Unlisted columns are unchanged.
    """
    return df.assign(**{col: df[col].mask(df[col].isin(vals)) for col, vals in codes.items()})
```

Per-column masking changes only what the assumption covers, and `assign` returns a new frame.

B7-L3 exercise 5

Hints. (1) Is it about rows, values or people? (2) Rows: structure. Values: coding. People: scope. (3) Completes are about who is in, not how answers are coded.

Answer.

- Grain: structure, checked with duplicated(keep=False) on the key.
- Region codes: coding, checked with isin(lookup).
- Age limit and completes only: scope, checked with a filter count that must be zero.

Naming the kind of assumption tells you the kind of check to write.

Common wrong answers.

- grain = `coding`: Grain is about how rows relate to each other. Check it with key uniqueness.
- region = `scope`: Valid region codes are a coding rule: every value must appear in the lookup.

- adults = `coding`: Age is coded correctly. The rule is about who is included: scope.